Thomas Liotier Lancelot Watelle FISEA

Notre analyse du projet :

Explication du diagramme UML :

Une classe *Game* contient toutes les fonctions principales ainsi que les fonctions utiles au déroulement du jeu.

Une classe *Entity*, dont héritent les classes *Player* et *Monster*, regroupe leurs attributs communs.

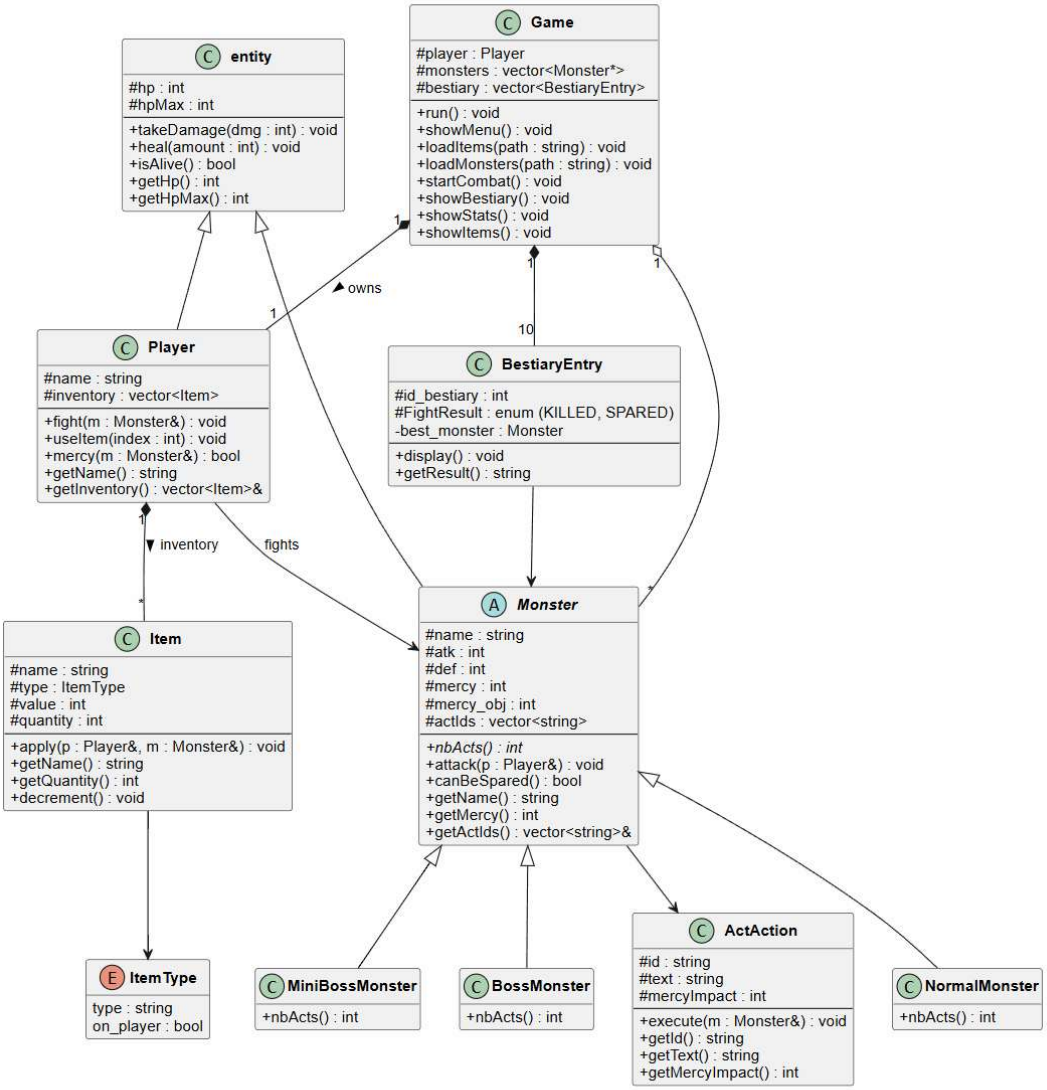
La classe *BestiaryEntry* contient une référence vers un *Monster*, ainsi que la manière dont le monstre a été vaincu (*Killed* ou *Spared*).

Nous parcourons le *bestiary* à la fin du jeu et à chaque manche afin de déterminer si le joueur adopte une approche « neutre », « génocidaire » ou « pacifiste ».

Pour les *items*, nous avons adopté une approche dynamique : un item possède un nom, un type, une valeur (positive ou négative) et une quantité. Il peut exister plusieurs types d'items.

Chaque type d'item est associé à une propriété *on_player*, qui permet de créer des objets de soin mais aussi d'attaque (on peut imaginer, par exemple, une grosse épée infligeant beaucoup de dégâts). Ainsi, si l'on définit un type *ATTACK* avec *on_player* = *false*, on comprend que la valeur de l'item devra être appliquée aux points de vie du monstre (par exemple : hp_monstre + (-100)).

Pour les types d'actions (*act*), nous allons définir une grande liste (par exemple 100 éléments). Ensuite, nous attribuerons à chaque monstre un certain nombre de ces éléments (indexés selon le type de monstre). Avec cette approche, on peut imaginer un *shuffle* des actions pour chaque monstre afin de diversifier le jeu.



La capture d’écran est présente dans le dossier de rendu ! Dans (schema_UML.jpg)

Exemple de nos CSV :

Items.csv:

	A	B	C	D
1	**nom**	**type**	**valeur**	**quantite**
2	Potion	HEAL	15	3
3	Snack	HEAL	8	5
4	SuperPotion	HEAL	30	1
5	Elixir	HEAL	50	1
6	PetitGateau	HEAL	5	8
7	BombePuante	DAMAGE	20	2
8	GrenadeDeChaussetteDAMAGE		35	1
9	PoudreDePoivre	DAMAGE	10	3

Monsters.csv:

	A	B	C	D	E	F	G	H	
1	**categorie**	**nom**	**hp**	**atk**	**def**	**mercyGoal**	**act1**	**act2**	**act3**
2	NORMAL	Froggit	30	7	1	100	COMPLIMENT	DISCUSS	-
3	NORMAL	Whimsun	25	5	1	100	JOKE	PET	-
4	NORMAL	Moldsmal	20	4	0	100	DANCE	COMPLIMENT	-
5	MINIBOSS	MimicBox	45	10	2	100	OBSERVE	PET	OFFER
6	MINIBOSS	Shyren	50	12	3	100	JOKE	DANCE	COMPI
7	BOSS	QueenByte	80	15	4	100	REASON	DANCE	JOKE
8	BOSS	Toriel	90	18	5	100	REASON	COMPLIMENT	PET

Exemple de notre code:

Pour le moment nous n'avons pas réellement codé les fonctions mais nous avons créé le squelette du projet (les .h possèdent les fonctions et les .cpp ont les fonctions créées mais pas de code). Voici certaines classes :

Class Monster :

Fichiers divers - Aucune configuration (Portée globale)

```
#pragma once
#include <string>
#include <vector>
#include "Entity.h"
#include "ActAction.h"
using std::string;
using std::vector;

class Player;

class Monster : public Entity {
protected:
    string name;
    int atk;
    int def;
    int mercy;
    int mercy_obj;
    vector<string> actIds;

public:
    Monster(int hp, int hpMax, const string& name, int atk, int def, int mercy_obj);
    virtual ~Monster() = default;

    virtual int nbActs() const = 0;
    void attack(Player& p);
    bool canBeSpared() const;
    string getName() const;
    int getMercy() const;
    vector<string>& getActIds();
};
```

Class Player :

Fichiers divers - Aucune configuration (Portée globale)

```cpp
#pragma once
#include <string>
#include <vector>
#include "Entity.h"
#include "Item.h"
using std::string;
using std::vector;

class Monster;

class Player : public Entity {
private:
    string name;
    vector<Item> inventory;

public:
    Player(int hp, int hpMax, const string& name);

    void fight(Monster& m);
    void useItem(int index);
    bool mercy(Monster& m);
    string getName() const;
    vector<Item>& getInventory();
};
```

Class entity :

Fichiers divers - Aucune configuration (Portée globale)

```cpp
#pragma once

class Entity {
protected:
    int hp;
    int hpMax;

public:
    Entity(int hp, int hpMax);
    virtual ~Entity() = default;

    void takeDamage(int dmg);
    void heal(int amount);
    bool isAlive() const;
    int getHp() const;
    int getHpMax() const;
};
```

Le reste du code est bien sur présent dans le dossier de code.